Exercise 4: Seeds, Sources, and Snapshots

Objective

Understand the difference between seeds and sources, define sources with freshness checks, and implement a **slowly changing dimension (SCD Type 2)** using dbt snapshots.

Prerequisites

- Exercise 3 completed (all models running, DuckDB database populated)

Part A: Sources

About Seeds vs Sources

	Seeds	Sources
What they are	CSV files committed to the dbt repo	Existing tables in your database (loaded by ELT tools)
Loaded by	`dbt seed`	External process (Fivetran, Airbyte, Airflow...)
Usage	Static reference data, small datasets	Raw production data
In production	Rarely	Always

In production, `raw_listings` would not be a CSV file you load — it would be a table populated continuously by an ELT pipeline (like the Airflow DAG from Session 8). A **source** tells dbt "this table exists in the database, but dbt did not create it".

Step 1: Define sources in YAML

Create `models/src/_sources.yml`:

```yaml
version: 2

sources:
  - name: airbnb_raw
    description: Raw Airbnb data loaded by the ingestion pipeline
    schema: dev
    tables:
      - name: raw_listings
        description: Airbnb property listings as ingested from the API
        loaded_at_field: updated_at
        freshness:
          warn_after: {count: 24, period: hour}
          error_after: {count: 48, period: hour}
```

```yaml
    - name: raw_hosts
      description: Airbnb host records
      loaded_at_field: updated_at
      freshness:
        warn_after: {count: 24, period: hour}
        error_after: {count: 48, period: hour}

    - name: raw_reviews
      description: Guest reviews
      loaded_at_field: review_date
      freshness:
        warn_after: {count: 48, period: hour}
        error_after: {count: 168, period: hour}
```

Key fields:

- `loaded_at_field` : the column dbt uses to measure freshness (usually a timestamp)
- `freshness.warn_after` : dbt warns if no new data in this window
- `freshness.error_after` : dbt fails if no new data in this window

Step 2: Update the staging models to use source()

Replace `ref('raw_listings')` with `source()` in `models/src/src_listings.sql` :

```sql
WITH raw AS (
    SELECT * FROM {{ source('airbnb_raw', 'raw_listings') }}
)

SELECT
    id                                      AS listing_id,
    name                                    AS listing_name,
    host_id,
    neighbourhood,
    room_type,
    minimum_nights,
    CAST(price AS DECIMAL(10, 2))           AS price_usd,
    CAST(created_at AS DATE)                AS created_at,
    CAST(updated_at AS DATE)                AS updated_at
FROM raw
```

Do the same for `src_hosts.sql` :

```sql
WITH raw AS (
    SELECT * FROM {{ source('airbnb_raw', 'raw_hosts') }}
)

SELECT
    id                                      AS host_id,
    name                                    AS host_name,
    CASE WHEN is_superhost = 't' THEN TRUE
```

```
        ELSE FALSE END                      AS is_superhost,
    CAST(created_at AS DATE)                AS created_at,
    CAST(updated_at AS DATE)                AS updated_at
FROM raw
```

And for `src_reviews.sql`:

```
WITH raw AS (
    SELECT * FROM {{ source('airbnb_raw', 'raw_reviews') }}
)

SELECT
    listing_id,
    reviewer_name,
    CAST(rating AS INTEGER)                 AS rating,
    CAST(review_date AS DATE)               AS review_date
FROM raw
```

Step 3: Run the models and verify the DAG

```
dbt run
```

Open the lineage graph:

```
dbt docs generate && dbt docs serve
```

Notice that the source nodes (`airbnb_raw.raw_listings` , etc.) now appear as **green database icons** at the root of the DAG — visually distinct from dbt models, because dbt did not create them.

Step 4: Check source freshness

```
dbt source freshness
```

dbt compares the `MAX(updated_at)` of each source table against the current time. Since our seed data has old dates, you will see a warning or error — which is exactly the point: this command tells you when your ingestion pipeline has stopped delivering data.

```
Found 3 sources to check freshness for
1 of 3 START freshness of airbnb_raw.raw_listings .... [RUN]
1 of 3 ERROR stale freshness of airbnb_raw.raw_listings  [ERROR in 0.03s]
```

In a CI/CD pipeline (like the GitHub Actions from Session 9), you would run this command before your dbt run to abort if source data is stale.

Part B: Snapshots

About Slowly Changing Dimensions (SCD Type 2)

Imagine a host changes from `is_superhost = false` to `is_superhost = true`. In a normal table, the old value is **overwritten** — you lose history.

SCD Type 2 solves this by adding rows instead of updating them:

host_id	is_superhost	dbt_valid_from	dbt_valid_to
102	false	2025-02-20	2025-11-01
102	true	2025-11-01	null

`dbt_valid_to = null` means "this is the current record". This lets you answer questions like "how many superhosts did we have in March 2025?" — impossible with a normal table.

Step 5: Create a snapshot

Create `snapshots/snap_hosts.sql`:

```
{% snapshot snap_hosts %}

{{
    config(
        target_schema='snapshots',
        unique_key='host_id',
        strategy='timestamp',
        updated_at='updated_at'
    )
}}

SELECT * FROM {{ source('airbnb_raw', 'raw_hosts') }}

{% endsnapshot %}
```

Config explained:

- `unique_key`: the column that identifies a record
- `strategy='timestamp'`: dbt detects changes by comparing the `updated_at` column
- `updated_at`: the column to check for changes

Step 6: Run the snapshot

```
dbt snapshot
```

dbt creates a `snapshots.snap_hosts` table with two extra columns per row: `dbt_valid_from` and `dbt_valid_to`.

Inspect the result:

```python
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("""
    SELECT host_id, host_name, is_superhost, dbt_valid_from, dbt_valid_to
    FROM snapshots.snap_hosts
    ORDER BY host_id
""").df())
conn.close()
```

You should see 4 rows, all with `dbt_valid_to = NULL` (all records are current).

Step 7: Simulate a source data change

Carlos Lopez (host 102) just became a superhost. His record in the raw table is updated. Simulate this by running the following Python script:

```python
import duckdb

conn = duckdb.connect("airbnb.duckdb")
conn.execute("""
    UPDATE raw_hosts
    SET is_superhost = 't',
        updated_at = '2025-11-01'
    WHERE id = 102
""")
conn.close()
print("Host 102 updated: is_superhost = true")
```

Run the script:

```
python update_host.py
```

Step 8: Run the snapshot again

```
dbt snapshot
```

Now inspect the snapshot table again:

```python
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("""
    SELECT host_id, host_name, is_superhost, dbt_valid_from, dbt_valid_to
    FROM snapshots.snap_hosts
    WHERE host_id = 102
    ORDER BY dbt_valid_from
```

```
""").df())
conn.close()
```

You should see **two rows** for host 102:

host_id	host_name	is_superhost	dbt_valid_from	dbt_valid_to
102	Carlos Lopez	false	2025-02-20	2025-11-01
102	Carlos Lopez	true	2025-11-01	null

dbt automatically:

1. Closed the old record by setting `dbt_valid_to = 2025-11-01`
2. Inserted a new record with `dbt_valid_from = 2025-11-01` and `dbt_valid_to = null`

Step 9: Query history with the snapshot

Answer business questions that are impossible without SCD Type 2:

```
import duckdb

conn = duckdb.connect("airbnb.duckdb")

print("How many superhosts existed on March 2025?")
print(conn.execute("""
    SELECT COUNT(*) AS superhost_count
    FROM snapshots.snap_hosts
    WHERE is_superhost = true
      AND dbt_valid_from <= '2025-03-01'
      AND (dbt_valid_to > '2025-03-01' OR dbt_valid_to IS NULL)
""").df())

conn.close()
```

In March 2025 Carlos was still `is_superhost = false`. Only Maria Garcia and Ana Fernandez were superhosts at that point.

Project Structure — Final State

```
airbnb/
├── dbt_project.yml
├── seeds/
│   ├── raw_listings.csv
│   ├── raw_hosts.csv
│   └── raw_reviews.csv
├── models/
│   ├── src/
│   │   ├── _sources.yml              ← Source definitions + freshness
│   │   ├── src_listings.sql          ← source() call
```

```
│   │   ├── src_hosts.sql             ← source() call
│   │   ├── src_reviews.sql           ← source() call
│   │   └── _int_listing_with_host.sql← ephemeral
│   └── marts/
│       ├── fct_listings.sql          ← table
│       └── fct_reviews_daily.sql     ← incremental
└── snapshots/
    └── snap_hosts.sql                ← SCD Type 2
```

Key Concepts

Concept	What it means
Source	External table not created by dbt. Declared with `source()`.
Freshness check	`dbt source freshness` — alerts when ingestion stops delivering data.
Snapshot	SCD Type 2 history table. dbt adds `dbt_valid_from`/`dbt_valid_to` columns.
SCD Type 2	Never overwrite records — insert new rows to track history.
`strategy='timestamp'`	Detects changes by comparing `updated_at` across runs.

Questions

1. What is the difference between calling `source('airbnb_raw', 'raw_listings')` and `ref('raw_listings')`? When would you use each?
2. A snapshot uses `strategy='check'` and `check_cols=['is_superhost', 'name']`. How is this different from `strategy='timestamp'`? When is it useful?
3. A junior analyst runs `UPDATE snap_hosts SET is_superhost = true WHERE host_id = 104` directly in the database. What problem does this create?
4. How would you use `snap_hosts` in a downstream mart model to always see the current state of each host?